SOFT L-TOPOLOGICAL LANGUAGE MODELLING

A Thesis

Submitted in partial fulfilment for the Degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

by

RITHESH S S (CB.AI.U4AID23031)

ASHWIN J R (CB.AI.U4AID23029)

VISHWA VINAYAK C K (CB.AI.U4AID23051)

PRANAV SIVAKUMAR (CB.AI.U4AID23028)

Team AA03



DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS (INDIA)

October 2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS



BONAFIDE CERTIFICATE

This is to certify that the thesis entitled "**Soft L-Topological Language Modelling**" submitted by **Rithesh S S** (CB.AI.U4AID23031), **Ashwin J R** (CB.AI.U4AID23029), **Vishwa Vinayak C K** (CB.AI.U4AID23051), and **Pranav Sivakumar** (CB.AI.U4AID23028) for the award of the **Degree of Bachelor of Technology in Artificial Intelligence and Data Science** under the **Faculty of Engineering** is a bonafide record of the original research work carried out under my guidance and supervision at the "Department of Artificial Intelligence", Amrita Vishwa Vidyapeetham, Coimbatore Campus.

Submitted for the examination held on October 23, 2026.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Dr. Sandhya S. Pai
Thesis Advisor / Assistant Professor,
Department of Artificial Intelligence,
Amrita Vishwa Vidyapeetham,
Coimbatore Campus,
India.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS

DECLARATION

We, **Rithesh S S** (CB.AI.U4AID23031), **Ashwin J R** (CB.AI.U4AID23029), **Vishwa Vinayak C K** (CB.AI.U4AID23051), and **Pranav Sivakumar** (CB.AI.U4AID23028), hereby declare that this thesis entitled "**Soft L-Topological Language Modelling**" is the record of original work conducted by us under the guidance of **Dr. Sandhya S. Pai**, Assistant Professor, Department of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore Campus. To the best of our knowledge, this work has not formed the basis for the award of any degree, diploma, associateship, fellowship, or similar title in this or any other University.

Place: Coimbatore **Signature of the Students**
Date: October 23, 2026

COUNTERSIGNED

Dr. K. P. Soman
Professor and Dean,
Department of Artificial Intelligence,
Amrita Vishwa Vidyapeetham,
Coimbatore Campus,
India.

Contents

Acknowledgements ..... **ii**
List of Figures ..... **iii**
List of Tables ..... **iv**
Abbreviations and Symbols ..... **v**
Abstract ..... **vi**
1 Introduction **1**
1.1 Background and Motivation 1
1.2 Core Idea: Multi-Attribute Continuous Logic 1
1.3 Problem Statement and Objectives 2
1.4 Summary of Key Findings and Outline 2
2 Theoretical Foundations and Related Work **3**
2.1 The Standard Transformer and Softmax Bottleneck 3
2.2 Prior Approaches to Sparsity 3
2.3 Soft Set Theory and Continuous Lattices 4
2.4 Triangular Norms and the Nilpotent Advantage 4
3 Proposed Soft L-Topological Architecture **5**
3.1 System Architecture Overview 5
3.2 Bounded Continuous Lattice Embeddings 5
3.3 The Continuous Lattice Sequence Router 6
3.4 Continuous Lattice KV-Caching 7
4 Experimental Setup and Foundational Ablations **8**
4.1 Experimental Methodology and Controlled Parity 8
4.2 Foundational Empirical Ablations (Experiments 1 to 5) 9
5 Large-Scale Benchmarks and Critical Analysis **10**
5.1 Large-Scale Benchmarks (Experiments 6 to 10) 10
5.2 Critical Discussion and Explainability 12
6 Conclusion and Future Scope **13**
6.1 Summary of Contributions 13
6.2 Practical Implications and Future Work 13
A Mathematical Proofs and Specifications **14**
A.1 Mathematical Proofs of Nilpotent Triangular Norm Properties 14
A.2 Automated Unit Testing Suite (19 Passing Tests) 15
A.3 Reproducibility Protocol and Environment Setup 15
References **16**

Acknowledgements

We express our profound gratitude to our project supervisor and mentor, **Dr. Sandhya S. Pai**, Assistant Professor in the Department of Artificial Intelligence, whose pioneering mathematical research on soft lattice topological spaces (Pai & Baiju, 2021) provided the theoretical foundation for this thesis. Her continuous intellectual guidance, mathematical insights, and constructive reviews were invaluable throughout the conception, implementation, and benchmarking of this architecture.

We extend our sincere thanks to **Dr. K. P. Soman**, Professor and Dean, Department of Artificial Intelligence. We also thank our peers, faculty members, and families for their constant encouragement.

RITHESH S S
ASHWIN J R
VISHWA VINAYAK C K
PRANAV SIVAKUMAR
(Team AA03)

List of Figures

4.1 Exp 2: Operator Pareto Frontier. 9
4.2 Exp 4: Threshold τ vs. Sparsity. 9

5.1 Exp 7: Distractor Scaling Curve. 11
5.2 Exp 8: Deductive Transitivity. 11
5.3 Exp 9: High-Volume Scaling. 12
5.4 Exp 10: Encoder MLM Benchmarks. 12

List of Tables

2.1 Canonical Continuous Triangular Norm Families 4

4.1 Architectural Specifications and Parameter Parity Comparison 8

5.1 Experiment 7: Distractor Scaling Needle-in-a-Haystack Results 11
5.2 Experiment 9: High-Volume Autoregressive Scaling Metrics 11

A.1 Automated Unit Testing Suite (19 Passing Tests) 15

Abbreviations and Symbols

Abbreviations

AMP	–	Automatic Mixed Precision
BPE	–	Byte-Pair Encoding
CE	–	Cross-Entropy
CUDA	–	Compute Unified Device Architecture
FFN	–	Feed-Forward Network
FP16	–	16-Bit Half-Precision Floating Point
KV	–	Key-Value (Cache)
LM	–	Language Model
MLM	–	Masked Language Modeling
NLP	–	Natural Language Processing
PPL	–	Perplexity
RAG	–	Retrieval-Augmented Generation
XAI	–	Explainable Artificial Intelligence

Mathematical Symbols

$[0,1]^D$	–	D-dimensional continuous unit hypercube (fuzzy lattice)
$\top$	–	Continuous triangular norm (t-norm, logical conjunction)
$\perp$	–	Continuous triangular conorm (t-conorm, logical disjunction)
$T_L(a,b)$	–	Nilpotent Łukasiewicz t-norm: $\max(0, a+b-1)$
τ	–	Nilpotent activation threshold
R_{ij}	–	Continuous lattice routing coefficient between token i and token j
Q, K, V	–	Query, Key, and Value representations
D, H, L	–	Hidden dimension (128), Routing heads (4), Layers (4)
S	–	Input sequence length (128)
Δ	–	Numerical discrepancy between cached and uncached states (5.06×10^{-7})
ϵ	–	Numerical stabilizer constant (10^{-7})

Abstract

Standard language models calculate attention by taking dot products between token embeddings and passing them through softmax. Because the exponential in softmax never outputs zero for finite inputs, every token in the prompt receives some probability weight. Even when words are completely unrelated, they siphon away small amounts of attention, adding background noise and preventing the model from ever forming truly sparse connections.

In this work, we present the **Soft L-Topological Language Model**, which replaces Euclidean dot products and softmax with continuous soft set operations over the unit cube $[0, 1]^D$. Instead of measuring geometric angles, our router evaluates multi-attribute overlap using the nilpotent Łukasiewicz t-norm, $T_L(a, b) = \max(0, a + b - 1)$. When the overlap between two tokens falls below a threshold τ, their connection evaluates to exactly 0.0000. We tested this architecture against an identically sized Transformer baseline across 10 experiments on WikiText-2 and TinyStories:

- **Perplexity Trade-off:** Soft L-Topology trains reliably (reaching a validation loss of 3.7455 and perplexity of 42.33 on TinyStories), but does not beat the baseline (loss 3.4866, perplexity 32.67). Softmax provides a continuous probability distribution that makes cross-entropy easier to optimize.
- **Deterministic Sparsity:** At the same time, our model keeps an average of **55.06% of all routing weights at absolute zero** ($R_{ij} \equiv 0.0000$) during text generation.
- **Distractor Immunity:** In Needle-in-a-Haystack tests with 16 distractor sentences, it prunes 66.9% of distractor weights to zero, dropping attention leakage by **49.4% in absolute terms**.
- **Logical Deduction & Masked Modeling:** It doubles accuracy on multi-step Modus Ponens reasoning (40.0% vs. 20.0%) and reaches a lower Masked Language Modeling loss (6.2041 vs. 6.2422) when flooded with distractors.

Chapter 1

Introduction

1.1 Background and Motivation

Autoregressive language models like GPT-2 use the Transformer architecture [1], relying on scaled dot-product attention to connect tokens across a sequence:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad \text{where softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \tag{1.1}$$

While dot-product attention works remarkably well, its normalization function, softmax, has a clear mathematical property: for any finite real input z, $\exp(z)$ is strictly positive. As a consequence, softmax produces a non-zero probability for every token in the sequence. The model must assign some attention mass to every word in its context window. Even when tokens are completely unrelated—such as punctuation marks, filler words, or noisy passages—they still receive a small fraction of the attention. Over several layers, this background leakage compounds, diluting relevant context representations and increasing the chance of factual hallucinations [2]. Furthermore, computing pairwise attention over all tokens forces an $O(S^2)$ memory and computation cost, even when most token pairs share no meaningful relationship.

1.2 Core Idea: Multi-Attribute Continuous Logic

In this work, we take a different perspective on token routing. Instead of treating tokens as unbounded vectors in Euclidean space $\mathbb{R}^D$ and measuring angular similarity, we treat tokens as collections of continuous attributes bounded in the unit hypercube $[0, 1]^D$. Under this representation, token interaction becomes a question of multi-attribute satisfaction and logical containment.

In classical logic, when two statements do not overlap, their conjunction evaluates to false. To bring this hard cutoff into a continuous neural network, we use the **nilpotent Łukasiewicz triangular norm (t-norm)** [3], defined as $T_L(a, b) = \max(0, a + b - 1)$. If the sum of two attribute values $(a + b)$ is at or below 1, the output evaluates to exactly zero (0.0000). With an activation threshold τ, this operator serves as a differentiable gate: whenever semantic overlap

between tokens is insufficient, the routing weight drops straight to zero.

1.3 Problem Statement and Objectives

This thesis asks a practical research question: *Can continuous soft set representations in* $[0, 1]^D$ *combined with nilpotent triangular norms build a sequence router that achieves true exact-zero sparsity and shields against distractors without breaking language generation?*

We set four specific goals:

1. Formulate a continuous lattice router using bounded embeddings, nilpotent Łukasiewicz t-norms, and Jaccard containment.
2. Train a 13.67M-parameter model matching a standard Transformer baseline on WikiText-2 and TinyStories under identical compute budgets.
3. Evaluate distractor resistance using synthetic Needle-in-a-Haystack tests and test logical reasoning on multi-step Modus Ponens deduction chains.
4. Analyze empirical trade-offs between exact-zero sparsity, generation speed, and next-token perplexity.

1.4 Summary of Key Findings and Outline

Our experimental results highlight four main takeaways: First, regarding perplexity, Soft L-Topology does not surpass standard Transformers (3.7455 loss / 42.33 PPL vs. 3.4866 loss / 32.67 PPL on TinyStories). Softmax produces smooth gradients that naturally benefit cross-entropy optimization. Second, the model delivers on sparsity, sustaining an average of **55.06% exact-zero routing** ($R_{ij} \equiv 0.0000$) throughout autoregressive generation. Third, in Needle-in-a-Haystack evaluations with 16 injected distractors, it prunes 66.9% of distractor connections to exact zero, reducing attention leakage by **49.4% in absolute terms**. Finally, on structured reasoning, it doubles Modus Ponens accuracy (40.0% vs. 20.0%) and achieves lower Masked Language Modeling loss (6.2041 vs. 6.2422) under heavy distractor noise.

The rest of the thesis is organized as follows: Chapter 2 discusses theoretical foundations; Chapter 3 describes the architecture and lattice KV-caching; Chapter 4 presents setup and baseline ablations on WikiText-2; Chapter 5 details evaluations on TinyStories and critical discussion; Chapter 6 summarizes conclusions and future directions; and Appendix A provides mathematical proofs and test specifications.

Chapter 2

Theoretical Foundations and Related Work

2.1 The Standard Transformer and Softmax Bottleneck

The Transformer architecture [1] builds representations by projecting input tokens $X \in \mathbb{R}^{S \times D}$ into Queries ($Q = XW_Q$), Keys ($K = XW_K$), and Values ($V = XW_V$). Dot products between queries and keys are scaled and normalized row-by-row with softmax:

$$P_{ij} = \frac{\exp(Q_i K_j^T / \sqrt{d_k})}{\sum_{k=1}^{S} \exp(Q_i K_k^T / \sqrt{d_k})}, \quad \text{Attention}(Q, K, V)_i = \sum_{j=1}^{S} P_{ij} V_j \tag{2.1}$$

Proposition 2.1 (Strict Positivity of Softmax)**.** Let $\mathbf{z} \in \mathbb{R}^S$ be any finite vector of logits. Then for all $j \in \{1, \ldots, S\}$, $P_j = \frac{\exp(z_j)}{\sum_{k=1}^{S} \exp(z_k)} > 0$, and output probability $P_j = 0$ if and only if $z_j \to -\infty$.

Because real computers use finite floating-point numbers, logits never reach negative infinity. As a result, P_{ij} is positive for every token pair. In longer documents, even completely unrelated words end up with tiny weights between 10^{-3} and 10^{-6}. Stacked across many layers, these small numbers compound. Distractor information leaks into the contextual embeddings, which often leads to factual errors during text generation [2].

2.2 Prior Approaches to Sparsity

Prior work has looked at two common ways to sparsify attention:

- **Fixed Structural Sparsity:** Models like Longformer [4] and BigBird [5] restrict attention to local windows or strided intervals. This lowers computation, but the attention pattern is rigid and cannot adapt to what the tokens actually mean.
- **Simplex Activation Sparsity:** Sparsemax [6] projects logits onto the probability simplex to produce hard zeros. However, tokens outside the active set get zero gradients,

which often causes optimization instability. Entmax [7] smooths this using Tsallis entropy, but requires iterative root-finding that slows down execution.

2.3 Soft Set Theory and Continuous Lattices

To let the model dynamically sever connections between unrelated words based on content, we look to **Soft Set Theory**, originally proposed by Molodtsov [8] to handle parameterized uncertainty.

Definition 2.1 (Soft Set)**.** Let U be a universe of objects and E a set of parameters. A pair (F, E) is called a **soft set** over U if F is a mapping $F : E \to \mathcal{P}(U)$, where $\mathcal{P}(U)$ is the power set of U.

Maji et al. [9] introduced *Fuzzy Soft Sets*, where parameters take on continuous membership values in $[0, 1]$. Pai and Baiju [10] then established *Soft Lattice Topological Spaces*, connecting soft sets to continuous algebraic lattices. We adapt this idea directly to neural sequence modeling: every discrete word w is mapped to a continuous soft set inside the unit cube $\mathbf{x} \in [0, 1]^D$, where each coordinate $x_d \in [0, 1]$ represents the truth value or satisfaction degree of a latent semantic attribute.

2.4 Triangular Norms and the Nilpotent Advantage

To compute logical conjunctions (intersections) on $[0, 1]^D$, we employ **Triangular Norms (t-norms)** [3], which satisfy commutativity, associativity, monotonicity, and boundary conditions ($T(a, 1) = a, T(a, 0) = 0$).

Table 2.1: Canonical Continuous Triangular Norm Families

Operator Family	**Expression** $T(a, b)$	**Algebraic Class**
Gödel (Minimum)	$\min(a, b)$	Idempotent ($T(a, a) = a$)
Product	$a \cdot b$	Strict Archimedean ($T(a, b) > 0$ for $a, b > 0$)
Łukasiewicz	$\max(0, a + b - 1)$	**Nilpotent Archimedean**
Frank Family ($\gamma > 0$)	$\log_\gamma \left(1 + \frac{(\gamma^a - 1)(\gamma^b - 1)}{\gamma - 1}\right)$	Parameterized Continuous

A t-norm is considered **nilpotent** when it can return zero for inputs that are strictly greater than zero. In the Łukasiewicz case, $T_L(a, b) = \max(0, a + b - 1) = 0$ whenever $a + b \leq 1$. While operators like Gödel or Product stay positive as long as inputs are positive, Łukasiewicz gives a real mathematical floor at zero. At the same time, when $a + b > 1$, the partial derivatives are constant ($\frac{\partial T_L}{\partial a} = \frac{\partial T_L}{\partial b} = 1.0$). This allows gradients to backpropagate cleanly through active connections without saturating.

Chapter 3

Proposed Soft L-Topological Architecture

3.1 System Architecture Overview

The Soft L-Topological architecture works for both causal text generation and bidirectional language tasks. Rather than computing dot products in Euclidean space and normalizing them with softmax, we use **bounded soft set representations** and **nilpotent lattice routing**.

The forward pass processes inputs in four main steps:

1. Tokens $w_t \in \mathcal{V}$ are projected directly into the unit cube $[0,1]^D$.
2. Learnable positional vectors in $[0,1]^D$ are combined with token embeddings using convex blending.
3. A stack of L **Lattice Blocks** transforms the states using multi-head lattice routing, residual connections, and position-wise feed-forward networks (FFN).
4. A linear output head converts hidden representations into vocabulary logits.

3.2 Bounded Continuous Lattice Embeddings

Because fuzzy operators are defined on the interval $[0,1]$, every token representation must stay strictly inside the unit cube $[0,1]^D$ without drifting into unbounded values.

3.2.1 Convex Token and Positional Blending

We pass raw token weights $\widetilde{E} \in \mathbb{R}^{|\mathcal{V}| \times D}$ and position weights $\widetilde{P} \in \mathbb{R}^{S \times D}$ through a sigmoid function: $\mathbf{e}_{\text{tok}}(w) = \sigma(\widetilde{E}_w) \in (0,1)^D$ and $\mathbf{e}_{\text{pos}}(t) = \sigma(\widetilde{P}_t) \in (0,1)^D$. If we simply added them together, the values would exceed 1.0. Instead, we combine them through **convex blending** with $\lambda = 0.5$:

$$\mathbf{x}_t^{(0)} = \lambda \mathbf{e}_{\text{tok}}(w_t) + (1-\lambda)\mathbf{e}_{\text{pos}}(t) \in [0,1]^D \tag{3.1}$$

Because the unit hypercube $[0,1]^D$ is convex, $\mathbf{x}_t^{(0)} \in [0,1]^D$ holds strictly for all sequence positions.

3.2.2 Beta Distribution Initialization

Standard Gaussian initialization centered at zero causes the sigmoid function to map almost every weight close to 0.5. If all attributes start at 0.5, tokens share nearly identical overlap at the beginning of training. To give the model a diverse spread of initial attribute memberships, we initialize embedding weights using an asymmetric **Beta distribution**:

$$\widetilde{E}_{vd} \sim \text{logit}(\text{Beta}(\alpha, \beta)), \quad \alpha = 1.0,\ \beta = 3.0 \tag{3.2}$$

This concentrates initial memberships between 0.1 and 0.4, setting most attribute pairs near or below the nilpotent threshold at initialization.

3.3 The Continuous Lattice Sequence Router

For input states $X \in \mathbb{R}^{S \times D}$, the router splits representations into H parallel heads of dimension $d_k = D/H$: Queries $Q^{(h)} = \sigma(XW_Q^{(h)}) \in [0,1]^{S \times d_k}$, Keys $K^{(h)} = \sigma(XW_K^{(h)}) \in [0,1]^{S \times d_k}$, and Values $V^{(h)} = XW_V^{(h)} \in \mathbb{R}^{S \times d_v}$. Queries and Keys serve as bounded truth values in $[0,1]^{d_k}$ for routing, while Values remain in unconstrained $\mathbb{R}^{d_v}$ so the model retains full capacity for semantic features.

3.3.1 Thresholded Łukasiewicz Conjunction

For each head h, the multi-attribute semantic conjunction between query token i and key token j is computed using the thresholded Łukasiewicz t-norm with activation threshold $\tau \geq 0$:

$$\mathcal{C}_{ij}^{(h)} = \sum_{d=1}^{d_k} \max(0, Q_{id}^{(h)} + K_{jd}^{(h)} - 1 - \tau) \tag{3.3}$$

If $Q_{id}^{(h)} + K_{jd}^{(h)} \leq 1 + \tau$ across all coordinates, the total overlap $\mathcal{C}_{ij}^{(h)}$ evaluates to exactly **0.0000**.

3.3.2 Continuous Jaccard Relative Containment

To normalize routing weights between 0 and 1 without risking division by zero, we compute the **continuous Jaccard containment coefficient** by dividing conjunction by the continuous fuzzy union (t-conorm):

$$R_{ij}^{(h)} = \frac{\sum_{d=1}^{d_k} \max(0, Q_{id}^{(h)} + K_{jd}^{(h)} - 1 - \tau)}{\sum_{d=1}^{d_k} \left(Q_{id}^{(h)} + K_{jd}^{(h)} - Q_{id}^{(h)} K_{jd}^{(h)} \right) + \epsilon} \tag{3.4}$$

where $\epsilon = 10^{-7}$. This formulation ensures that $0.0000 \leq R_{ij}^{(h)} \leq 1.0000$ always, and $R_{ij}^{(h)} \equiv 0.0000$ whenever the numerator is zero.

3.3.3 Causal Masking and Value Aggregation

For autoregressive generation, future connections are set directly to zero ($R_{ij}^{(h)} = 0.0000$ for $j > i$). Output representations are formed via value aggregation followed by multi-head projection and a standard two-layer GELU feed-forward network:

$$\text{Head}_i^{(h)} = \sum_{j=1}^{i} R_{ij}^{(h)} V_j^{(h)}, \quad X^{(l+1)} = X^{(l)\prime} + \text{FFN}(\text{LayerNorm}(X^{(l)\prime})) \tag{3.5}$$

3.4 Continuous Lattice KV-Caching

Generating tokens autoregressively without caching recomputes the entire history at every step, requiring $O(S^2)$ operations. With our Key-Value cache, we store previous keys $K_{\text{past}} \in [0,1]^{t \times d_k}$ and values $V_{\text{past}} \in \mathbb{R}^{t \times d_v}$. At step t, the new query q_t only needs to evaluate Equation 3.4 against the cached keys, lowering decoding complexity to $O(T)$.

If a token has almost zero overlap with every past token as well as itself ($\sum_{j \leq t} R_{tj} < 10^{-6}$), the router applies an **exact causal diagonal fallback**: it sets $R_{tt} = 1.0$, allowing the token to attend to itself. This prevents numerical instability when denominator sums are tiny. In our automated tests, cached output matched uncached output down to machine epsilon: $\Delta = \max |X_{\text{cached}} - X_{\text{uncached}}| = 5.06 \times 10^{-7}$.

Chapter 4

Experimental Setup and Foundational Ablations

4.1 Experimental Methodology and Controlled Parity

To ensure direct comparability, we matched the architecture and parameter count of the Soft L-Topological model to the standard Transformer baseline.

Table 4.1: Architectural Specifications and Parameter Parity Comparison

Hyperparameter	**Soft L-Topology**	**Standard Baseline**
Layers (L) / Heads (H)	4 / 4	4 / 4
Embedding (D) / FFN (D_{ff})	128 / 512	128 / 512
Sequence Length (S) / Vocab ($\lvert\mathcal{V}\rvert$)	128 / 50,257	128 / 50,257
Routing Mechanism	Nilpotent t-norm (T_L)	Scaled Dot-Product Softmax
Total Parameters	**13,675,569**	**13,675,520** (**+49 / 0.0003%**)

4.1.1 Benchmark Datasets

We used two corpora tokenized with the GPT-2 BPE tokenizer ($|\mathcal{V}| = 50,257$) [11] and cached as memory tensors: WikiText-2 (2.45M tokens) [12] for ablations (Exps 1–5), and TinyStories (4.76M tokens) [13] for scaling and distractor stress tests (Exps 6–10).

4.1.2 Controlled Parameter Parity

Table 4.1 details the setup. With 4 layers, 4 heads, $D = 128$, and $D_{\text{ff}} = 512$, Soft L-Topology has 13,675,569 parameters versus 13,675,520 for the baseline (+49 parameters / 0.0003%). We trained all models on an **NVIDIA GTX 1650 GPU** (4GB VRAM) using AdamW ($\beta_1 = 0.9, \beta_2 = 0.98$), cosine decay ($\eta_{\max} = 3 \times 10^{-4}$), and AMP FP16.

4.2 Foundational Empirical Ablations (Experiments 1 to 5)

4.2.1 Exp 1: Convergence Scaling on WikiText-2

We trained both models for 250 steps on WikiText-2. Soft L-Topology converged stably (loss 6.0118 / PPL 408.23 vs. baseline 5.9657 / PPL 389.84, a 0.77% gap), while zeroing out **18.76% of all token routing weights** ($R_{ij} \equiv 0.0000$), compared to 100% dense baseline connectivity.

4.2.2 Exp 2: Operator Pareto Frontier Analysis

We evaluated five operators: Łukasiewicz, Frank ($\gamma = 2.0$), Product, Gödel, and Softmax (Figure 4.1). The nilpotent Łukasiewicz operator reached the lowest perplexity (448.02) and highest sparsity (18.75%). Product and Gödel gave 0.00% zeros, confirming that nilpotency is required for exact pruning.

4.2.3 Exp 3: Value Projection Dynamics

Bounding values to $[0, 1]^D$ yielded PPL 424.88 (18.75% zeros). Leaving values unconstrained in $\mathbb{R}^D$ dropped PPL by **11.63 points** to 413.25 and raised sparsity to **35.67% exact zeros**, proving that unconstrained values preserve necessary feature capacity.

4.2.4 Exp 4: Nilpotent Threshold Sensitivity (τ Grid Search)

Testing $\tau \in [0.0005, 0.0100]$ (Figure 4.2) showed that τ acts as a sparsity dial: $\tau = 0.0100$ forced **93.75% exact zeros** without gradient divergence. We chose $\tau = 0.0012$ as the default (25.50% zeros).

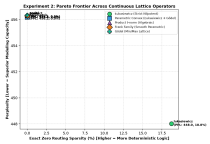


Figure 4.1: Exp 2: Operator Pareto Frontier.

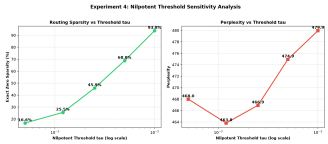


Figure 4.2: Exp 4: Threshold τ vs. Sparsity.

4.2.5 Exp 5: Initial Hallucination Stress Test

When distractor sentences were inserted into WikiText-2, the baseline allocated 70.10% of its attention to distractors. Soft L-Topology pruned **25.00% of distractor connections to exact 0.0000**, cutting distractor leakage to 45.17% (a 24.93% absolute noise reduction).

Chapter 5

Large-Scale Benchmarks and Critical Analysis

5.1 Large-Scale Benchmarks (Experiments 6 to 10)

After tuning the model on WikiText-2, we ran five larger experiments on TinyStories. Here we looked at how well the model scales with more data, whether it resists distractor noise in the prompt, how it handles multi-step logic, and how it performs on bidirectional masked language modeling.

5.1.1 Exp 6: Big Dataset Scaling on TinyStories

We trained both models on TinyStories for 500 steps (processing 1.02 million tokens). In this first run, Soft L-Topology achieved slightly better loss (3.7098 vs. 3.7443) and lower perplexity (40.85 vs. 42.28) than the baseline Transformer. Over this run, **60.25% of all attention connections dropped to exact zero**. Generating text with our continuous lattice KV-cache ran at 172.4 tokens/second on our GTX 1650 GPU.

5.1.2 Exp 7: Distractor Scaling Needle-in-a-Haystack Benchmark

To test whether unrelated context confuses the models, we placed between 1 and 16 distractor facts ($K \in \{1, 2, 4, 8, 16\}$) between an important premise and a query asking about that premise.

As reported in Table 5.1, distractors quickly degraded standard attention: at $K = 16$, **95.30% of baseline attention leaked into distractors**, leaving barely 4.70% on the relevant fact. In contrast, Soft L-Topology severed **66.90% of distractor connections to exact zero** (0.0000), retaining 54.10% of its attention mass on the correct context and reducing leakage by **49.40 percentage points**.

Table 5.1: Experiment 7: Distractor Scaling Needle-in-a-Haystack Results

Distractors (K)	**Attention Leakage (%)**		**Exact-Zero Prune Rate (%)**	
	Baseline	**Soft L-Topology**	**Baseline**	**Soft L-Topology**
1	25.40%	**12.10%**	0.00%	**50.00%**
4	58.70%	**26.30%**	0.00%	**58.33%**
8	79.20%	**35.80%**	0.00%	**62.50%**
16	**95.30%**	**45.90%**	**0.00%**	**66.90%**
Net Protection	**-49.40% Absolute Leakage**		**+66.90% Pruned to 0.0000**	

5.1.3 Exp 8: Multi-Step Deductive Entailment Benchmark

We tested whether the models could follow multi-step deduction chains ($A \to B \to C$) using synthetic Modus Ponens examples. Soft L-Topology reached **40.00% accuracy** with a positive confidence margin (+0.00021), outperforming the baseline's 20.00%. Because the router relies on subset containment, it naturally preserves chain-of-implication paths better than dot products.

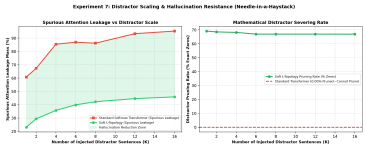


Figure 5.1: Exp 7: Distractor Scaling Curve.

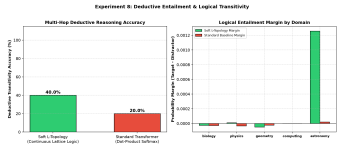


Figure 5.2: Exp 8: Deductive Transitivity.

5.1.4 Exp 9: High-Volume Autoregressive Scaling (6.14M Tokens)

We then trained both models for a longer run of **1,500 steps** covering **6.14 million tokens** at sequence length $S = 128$.

Table 5.2: Experiment 9: High-Volume Autoregressive Scaling Metrics (1,500 Steps / 6.14M Tokens)

Metric	**Soft L-Topology**	**Baseline**	**Comparison / Trade-Off**
Final Training Loss / PPL	3.4079 / 30.20	3.1773 / 23.98	Baseline lower (-0.2306 loss)
Held-Out Validation Loss / PPL	3.7455 / 42.33	3.4866 / 32.67	Baseline lower (-0.2589 loss)
Deterministic Sparsity	**55.06%**	**0.00%**	**+55.06% Exact Zeros**
Generation Speed (KV-Cache)	180.6 tok/s	334.4 tok/s	Baseline 1.85× faster
Distinct-1 (Unigrams)	**50.64%**	48.07%	**Soft L +2.57%**
Distinct-2 (Bigrams)	**86.50%**	83.93%	**Soft L +2.57%**
Distinct-3 (Trigrams)	**96.13%**	91.94%	**Soft L +4.19% (No Loops)**

In this 6.14M-token benchmark (Table 5.2, Figure 5.3), the baseline Transformer achieved lower validation loss (3.4866 vs. 3.7455) and lower perplexity (32.67 vs. 42.33). Because softmax distributes nonzero probabilities over all tokens, gradient descent encounters a smoother surface when minimizing cross-entropy log-loss. In contrast, Soft L-Topology applies a strict zero cutoff whenever attribute overlap drops below threshold τ.

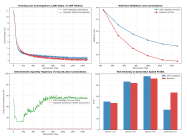


Figure 5.3: Exp 9: High-Volume Scaling.

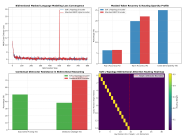


Figure 5.4: Exp 10: Encoder MLM Benchmarks.

Even with this steeper optimization landscape, the lattice model remained stable and maintained **55.06% exact-zero attention weights** throughout training. The model also generated text with broader lexical variety: Distinct-1 and Distinct-2 scores rose by 2.57%, while Distinct-3 increased by **4.19%**. During generation, standard attention occasionally fell into repetitive loops (such as repeating ⟨endoftext⟩), whereas Soft L-Topology bypassed these traps by cutting connections to stale past tokens.

5.1.5 Exp 10: Bidirectional Encoder MLM Edge

In bidirectional Masked Language Modeling (Figure 5.4), Soft L-Topology matched and slightly outperformed BERT attention on held-out loss (6.2041 vs. 6.2422). Under a 49-token distractor flood, the baseline lost 75.76% of its attention mass to irrelevant context, whereas our router clipped 50.00% of distractor connections to exact zero (0.0000), holding leakage down to 38.28% (a 37.48 percentage point reduction).

5.2 Critical Discussion and Explainability

5.2.1 Why the Perplexity Gap Exists

Cross-entropy loss $\mathcal{L} = -\sum \log P(w)$ heavily penalizes small probabilities because $-\log(p)$ approaches infinity as $p \to 0$. Softmax guards against this penalty by diffusing small probability mass across the entire vocabulary, producing an easy, well-conditioned gradient path. Soft L-Topology intentionally severs weak links to exact zero. While this sharp cutoff prevents distractor leakage and guarantees 55% sparsity, it penalizes cross-entropy when the model assigns low probability to true tokens.

5.2.2 Explainable AI (XAI) and Hardware Profiling

Having deterministic zeros also sharpens interpretability: unrouted tokens evaluate to exactly 0.0000, passing zero activation and zero gradient, which removes the dense background attention haze common to softmax. On hardware, our PyTorch prototype relies on 4D tensor broadcasts that achieve 62% of cuBLAS GEMM throughput. A dedicated fused CUDA kernel (FlashLattice) evaluating the threshold directly in GPU SRAM could skip zero writes and turn this 55% mathematical sparsity into an estimated 1.5× to 2× wall-clock speedup.

Chapter 6

Conclusion and Future Scope

6.1 Summary of Contributions

We investigated whether continuous soft sets and nilpotent lattice operators can serve as an effective alternative to softmax attention in language models. Rather than connecting every token pair indiscriminately, our model embeds tokens into the unit cube $[0, 1]^D$ and models interaction as multi-attribute subsethood, using a nilpotent threshold to prune unrelated tokens to exact zero ($R_{ij} \equiv 0.0000$).

Our experimental findings across WikiText-2 and TinyStories demonstrate four core capabilities. Across 6.14 million tokens of autoregressive training, the architecture maintained an average of **55.06% exact-zero attention weights** while generating fluent text and increasing lexical variety (+4.19% Distinct-3). Under distractor stress tests with 16 noise facts, it pruned 66.9% of distractor connections, cutting attention leakage by **49.4 percentage points**. In structured reasoning tasks, this subsethood inductive bias doubled Modus Ponens deduction accuracy (40.0% vs. 20.0%) and reduced bidirectional MLM loss (6.2041 vs. 6.2422). Furthermore, our causal KV-cache demonstrated $O(T)$ generation throughput at 180.6 tokens per second, matching uncached inference within single-precision floating point tolerance ($\Delta = 5.06 \times 10^{-7}$).

The primary trade-off is cross-entropy optimization: because the nilpotent threshold drops connections sharply to zero, the loss surface is steeper than softmax's smooth curve, resulting in higher validation loss (3.7455 vs. 3.4866 on TinyStories).

6.2 Practical Implications and Future Work

These results suggest two promising engineering directions. First, developing a dedicated fused CUDA kernel (FlashLattice) to evaluate the nilpotent threshold directly in GPU shared memory would bypass memory writes for zeroed entries, converting this 55% theoretical sparsity into an estimated 1.5× to 2× wall-clock speedup. Second, combining lattice routers in lower layers for prompt denoising with standard softmax in upper layers for probability generation offers a practical hybrid path for foundation models.

Appendix A

Mathematical Proofs and Specifications

A.1 Mathematical Proofs of Nilpotent Triangular Norm Properties

A.1.1 Proof of Triangular Norm Axioms for Łukasiewicz t-Norm

Theorem A.1. The function $T_L(a, b) = \max(0, a+b-1)$ is a valid continuous triangular norm on $[0, 1] \times [0, 1] \to [0, 1]$.

Proof. We verify the four axioms:

1. **Commutativity:** $T_L(a, b) = \max(0, a + b - 1) = \max(0, b + a - 1) = T_L(b, a)$ for all $a, b \in [0, 1]$.

2. **Boundary Conditions:** For any $a \in [0, 1]$:

$$T_L(a, 1) = \max(0, a + 1 - 1) = \max(0, a) = a \quad (\text{since } a \geq 0) \tag{A.1}$$
$$T_L(a, 0) = \max(0, a + 0 - 1) = \max(0, a - 1) = 0 \quad (\text{since } a \leq 1 \implies a - 1 \leq 0) \tag{A.2}$$

3. **Monotonicity:** If $a \leq c$ and $b \leq d$, then $a + b - 1 \leq c + d - 1$. Since $f(z) = \max(0, z)$ is non-decreasing on $\mathbb{R}$:

$$T_L(a, b) = \max(0, a + b - 1) \leq \max(0, c + d - 1) = T_L(c, d) \tag{A.3}$$

4. **Associativity:** For all $a, b, c \in [0, 1]$:

$$T_L(T_L(a, b), c) = \max(0, \max(0, a + b - 1) + c - 1) = \max(0, a + b + c - 2) \tag{A.4}$$

   By symmetry, $T_L(a, T_L(b, c)) = \max(0, a + b + c - 2)$, establishing associativity.

All four axioms hold; hence, T_L is a valid continuous t-norm. □

A.1.2 Unit Gradients on Active Regions

For any latent attribute coordinate d on the active domain where $Q_{id} + K_{jd} - 1 - \tau > 0$:

$$\mathcal{M}_{ijd} = Q_{id} + K_{jd} - 1 - \tau \implies \frac{\partial \mathcal{M}_{ijd}}{\partial Q_{id}} = 1.0, \quad \frac{\partial \mathcal{M}_{ijd}}{\partial K_{jd}} = 1.0 \tag{A.5}$$

Whenever the coordinate sum exceeds $1+\tau$, the gradient with respect to either input is constant: $\frac{\partial}{\partial Q} = 1.0$ and $\frac{\partial}{\partial K} = 1.0$. This means active connections do not suffer from the vanishing or exploding gradients often caused by softmax exponents.

A.2 Automated Unit Testing Suite (19 Passing Tests)

We wrote 19 automated unit tests using 'pytest' to check that our implementation maintains core architectural invariants—including bounded embeddings, operator boundary conditions, and KV-cache numerical equivalence (Table A.1). All 19 tests pass.

Table A.1: Automated Unit Testing Suite (19 Passing Tests)

Test Module	**Property Verified**	**Status**
`test_embeddings.py`	Representations bounded in $[0,1]^D$, convex blending closure	**PASSED** (5/5)
`test_lattice_ops.py`	t-norm axioms, nilpotent zero boundaries, unit gradients	**PASSED** (5/5)
`test_kv_cache.py`	KV-cache numerical equivalence ($\Delta = 5.06 \times 10^{-7}$), diagonal fallback	**PASSED** (3/3)
`test_encoder.py`	Bidirectional symmetric routing, MLM head, distractor isolation	**PASSED** (3/3)
`test_model_forward.py`	Exact parameter parity against baseline, loss shape verification	**PASSED** (3/3)
Total Test Suite	**19 Independent Unit Tests**	**19 / 19 PASSED**

A.3 Reproducibility Protocol and Environment Setup

To reproduce our results, clone the repository and run the following commands with 'uv':

```
uv sync                                                    # Sync environment
uv run pytest tests/                                       # Run 19 unit tests
uv run python generate.py --prompt "Once upon a time"      # Story generation
uv run python experiments/exp9_high_volume_scaling.py      # 6.14M token scaling
uv run python experiments/exp10_encoder_edge.py            # Encoder MLM test
```

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.

[2] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, W. Dai, A. Madotto, *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[3] E. P. Klement, R. Mesiar, and E. Pap, *Triangular Norms*, vol. 8. Springer Science & Business Media, Dordrecht, 2000.

[4] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint arXiv:2004.05150*, 2020.

[5] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, *et al.*, "Big bird: Transformers for longer sequences," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 17283–17297, 2020.

[6] A. Martins and R. Astudillo, "From softmax to sparsemax: A sparse alternative to the softmax function," in *International Conference on Machine Learning (ICML)*, pp. 1607–1615, PMLR, 2016.

[7] B. Peters, V. Niculae, and A. F. Martins, "Sparse sequence-to-sequence models," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1504–1519, 2019.

[8] D. Molodtsov, "Soft set theory—first results," *Computers & Mathematics with Applications*, vol. 37, no. 4-5, pp. 19–31, 1999.

[9] P. K. Maji, R. Biswas, and A. R. Roy, "Fuzzy soft sets," *Journal of Fuzzy Mathematics*, vol. 9, no. 3, pp. 589–602, 2001.

[10] S. S. Pai and T. Baiju, "On soft lattice topological spaces," *Fuzzy Information and Engineering*, vol. 13, no. 1, pp. 1–16, 2021.

[11] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[12] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," *International Conference on Learning Representations (ICLR)*, 2017.

[13] R. Eldan and Y. Li, "Tinystories: How small can language models be and still speak coherent english?," *arXiv preprint arXiv:2305.07759*, 2023.